

```

# Import Required Libraries
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST Dataset
(x_train, _), (_, _) = keras.datasets.mnist.load_data()

# Preprocess the Images
x_train = x_train.astype("float32")
x_train = (x_train - 127.5) / 127.5      # Normalize to [-1,1]
x_train = x_train.reshape(x_train.shape[0], 28, 28, 1)

# Training Parameters
batch_size = 64
noise_dim = 100
epochs = 10

# Create Dataset Batches
dataset = tf.data.Dataset.from_tensor_slices(x_train).shuffle(60000).batch(batch_size)

# Build Generator Model
generator = keras.Sequential([
    keras.layers.Input(shape=(noise_dim,)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(28 * 28 * 1, activation='tanh'),
    keras.layers.Reshape((28, 28, 1))
])

# Build Discriminator Model
discriminator = keras.Sequential([
    keras.layers.Input(shape=(28, 28, 1)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])

# Define Loss Function and Optimizers
loss_fn = keras.losses.BinaryCrossentropy()

g_optimizer = keras.optimizers.Adam(learning_rate=0.0002)
d_optimizer = keras.optimizers.Adam(learning_rate=0.0002)

# Training Loop
for epoch in range(epochs):
    for real_images in dataset:
        batch_size_now = real_images.shape[0]

        real_labels = tf.ones((batch_size_now, 1))
        fake_labels = tf.zeros((batch_size_now, 1))

        # -----
        # Train Discriminator
        # -----
        noise = tf.random.normal((batch_size_now, noise_dim))
        fake_images = generator(noise, training=True)

        with tf.GradientTape() as tape:
            real_output = discriminator(real_images, training=True)
            fake_output = discriminator(fake_images, training=True)

            real_loss = loss_fn(real_labels, real_output)
            fake_loss = loss_fn(fake_labels, fake_output)

            d_loss = real_loss + fake_loss

        d_gradients = tape.gradient(d_loss, discriminator.trainable_variables)

        d_optimizer.apply_gradients(
            zip(d_gradients, discriminator.trainable_variables)
        )

        # -----
        # Train Generator
        # -----
        noise = tf.random.normal((batch_size_now, noise_dim))

```

```

with tf.GradientTape() as tape:

    generated_images = generator(noise, training=True)

    output = discriminator(generated_images, training=True)

    g_loss = loss_fn(real_labels, output)

    g_gradients = tape.gradient(g_loss, generator.trainable_variables)

    g_optimizer.apply_gradients(
        zip(g_gradients, generator.trainable_variables)
    )

print(f"Epoch [{epoch+1}/{epochs}] D Loss: {d_loss.numpy():.4f} G Loss: {g_loss.numpy():.4f}")

# Generate Images After Training
noise = tf.random.normal((16, noise_dim))
generated_images = generator(noise, training=False)

# Display Generated Images
fig, axes = plt.subplots(4, 4, figsize=(6,6))

for i, ax in enumerate(axes.flat):
    ax.imshow((generated_images[i, :, :, 0] + 1) / 2, cmap='gray')
    ax.axis("off")

plt.tight_layout()
plt.show()

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 — 0s 0us/step

```

Epoch [1/10] D Loss: 0.5258 G Loss: 1.5112
Epoch [2/10] D Loss: 0.5674 G Loss: 1.7248
Epoch [3/10] D Loss: 0.6360 G Loss: 1.4879
Epoch [4/10] D Loss: 0.2355 G Loss: 2.5067
Epoch [5/10] D Loss: 0.9248 G Loss: 1.3785
Epoch [6/10] D Loss: 0.9279 G Loss: 1.5202
Epoch [7/10] D Loss: 0.7442 G Loss: 1.6702
Epoch [8/10] D Loss: 1.3684 G Loss: 1.5293
Epoch [9/10] D Loss: 0.4458 G Loss: 2.0595
Epoch [10/10] D Loss: 0.4128 G Loss: 2.0723

```



